COMP 182 Final Exam Study Guide

Lectures 8 through 13 | 52 practice questions | No coding

Name _____ Date _____

Practice the final exam question styles: choose one multiple-choice answer, complete operation traces, list traversals, and draw trees and heaps. Show intermediate steps. Stack entries are bottom to top; queues are front to rear. Count tree height in edges: a leaf has height 0 and an empty tree has height -1. Use separate paper for extra drawing space.

Lecture 8 Stacks

1. Stack operations

An empty stack receives push(3), push(5), peek(), pop(), push(8). Which gives the values returned by peek and pop, followed by the final stack?

A. 3, 5; stack 3, 8

B. 5, 5; stack 3, 8

C. 5, 3; stack 5, 8

D. 5, 5; stack 8

2. Infix to postfix with an operator stack

Convert (3 + 4) / (12 + 13 / 7) to postfix. Show the operator stack and cumulative output after every token. At END, flush any remaining operators. All binary operators are left-associative; multiplication and division have higher precedence than addition and subtraction.

Token	Operator stack bottom to top	Postfix output
(		
3		
+		
4		
)		
/		
(		
12		
+		
13		
/		
7		
)		
END		

Lecture 8 Stacks and Expressions

3. Postfix evaluation

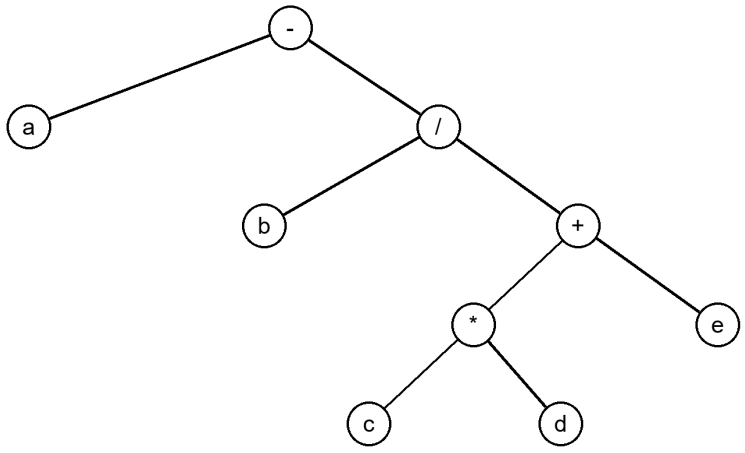
Evaluate 12 4 + 6 6 2 / + * using integer arithmetic. Show the operand stack after each token. Identify which popped item is the right operand for division.

Token	Operand stack bottom to top
12	
4	
+	
6	
6	
2	
/	
+	
*	

Result _____ Right-operand rule _____

4. Read an expression tree

The circles below contain operators and operands. Write the fully parenthesized infix expression and its postfix form. Explain in one sentence why an expression tree need not satisfy the BST ordering rule.



Expression tree for Question 4

Infix _____

Postfix _____

Lecture 8 Stack Applications

5. Brace matching trace

Ignore non-brace characters. Trace {a{b}c}, showing the brace stack after each character. Then trace }{ and identify the first failure. State both the condition required at a closing brace and the condition required at the end.

Character	Brace stack or failure
{	
a	
{	
b	
}	
c	
}	

---

6. Stack operation costs

Which cost statement is correct for a stack with its top at the end of a resizable array?

A. Every push is worst-case O(1), including resizing.

B. Every pop requires shifting all remaining entries.

C. Push is amortized O(1); a resizing push may take O(n).

D. Peek requires a search through the array.

7. Reverse text with a stack

Push the characters of TREE from left to right onto an empty stack, then pop until empty. Give the stack after all pushes, the pop sequence, and the resulting text. Do not write a method.

All pushes complete bottom to top	Pop sequence	Resulting text

8. Brace validation

Why is checking only the final stack size insufficient for brace matching?

A. It does not detect an unmatched closing brace that occurs before any opening brace.

B. The stack must contain all non-brace characters.

C. Matching braces requires sorting the input.

D. An empty final stack always means every prefix is valid.

Lecture 9 Queue Tracing

9. FIFO operations

Trace an initially empty FIFO queue. Record each returned item, if any, and the queue after each operation. Offer operations need no item-return entry.

Operation	Returned item	Queue front to rear
offer(20)		
offer(10)		
offer(30)		
remove()		
peek()		
offer(40)		
poll()		

10. Empty queue behavior

For an empty ArrayDeque used as a queue, which statement is correct?

A. poll and remove both return null.

B. peek and element both throw an exception.

C. remove returns null and poll throws an exception.

D. poll and peek return null; remove and element throw an exception.

11. Circular array queue

Capacity is 5, front = 3, back = 0, size = 3. A is at index 3, B at 4, and C at 0. The back index identifies the last stored item. Perform the operations below. Use size to distinguish full and empty states.

Operation	Front	Back	Size	Logical order
remove				
offer D				
offer E				
remove				

12. One-node circular linked queue

A circular linked queue stores a reference to its last node. Draw the one-node state, including the next link. Mark where the front is. Then describe the last-node reference and size after removing that only item.

Lecture 9 Queue Concepts

13. Queue API

A queue uses a private linked list as backing storage. Which design best preserves its FIFO interface?

A. Allow clients to remove any indexed element.

B. Insert at the rear and remove at the front through queue operations.

C. Sort the linked list after each insertion.

D. Expose the internal list so clients can rearrange it.

14. Compare queue costs

Fill in the usual enqueue and dequeue costs in terms of n, the number of stored items. Assume a linked queue has front and rear references, the fixed circular array has available capacity, and the shifting array has available capacity.

Representation	Enqueue cost	Dequeue cost
Linked queue with front and rear		
Fixed circular array		
Array that shifts after dequeue		

15. Remove all items in queue order

A queue contains red, blue, green from front to rear. Repeatedly remove and report an item until the queue is empty. Complete the trace. State the stopping condition in words; do not write code.

Removal number	Returned item	Remaining queue
1		
2		
3		

---

16. Enqueue into a circular linked queue

A nonempty circular linked queue has a last-node reference L, and L.next identifies the front. A detached node N holds a new rear item. Which sequence preserves the existing front?

A. Make N.next point to null and leave L unchanged.

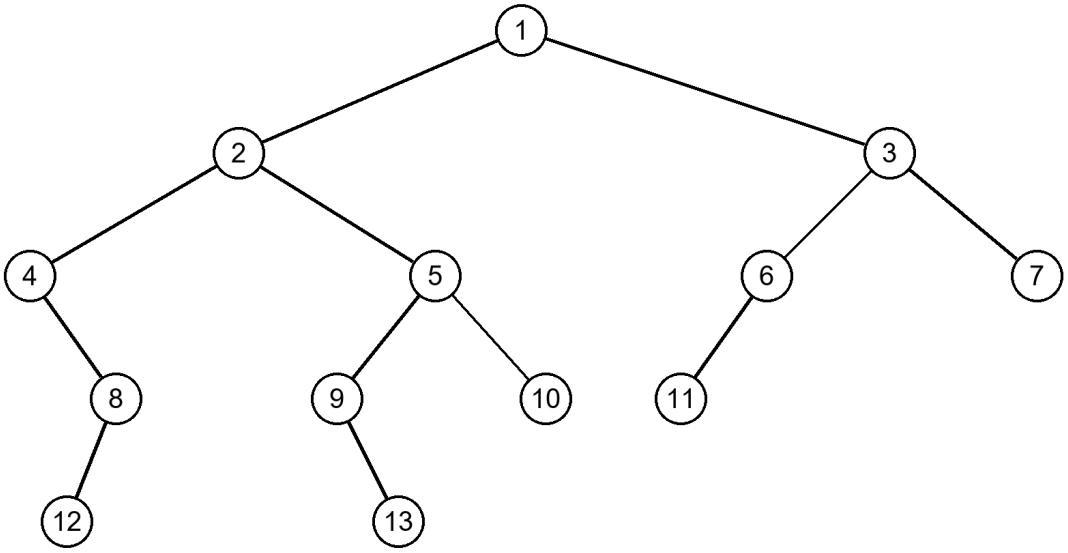
B. Make L.next point to N, then set N.next to L.next.

C. Make N.next point to the old front, make L.next point to N, then make N the last node.

D. Make N the front and discard the old last node.

Lecture 10 BST Node Numbers

Use Tree A for Questions 17-19. Circles show node numbers only, not stored keys. Follow the left and right links; do not compare node numbers. Hidden keys obey the BST ordering rule.



Tree A practice BST with omitted key values

17. Tree vocabulary

Identify the root, all leaves, the ancestors of node 13, and the height in edges. Also state the number of nodes and levels.

18. BST search cost

Which statement correctly describes ordinary, unbalanced BST search?

A. It is O(1) for every tree.

B. It follows one downward path, taking O(h + 1) time; a chain can require O(n).

C. It visits every node in preorder before deciding.

D. It is guaranteed O(log n) without balancing.

19. Three traversal orders

List all node numbers in preorder, inorder, and postorder. Always visit the left subtree before the right subtree. Do not substitute hidden key values.

Preorder _____

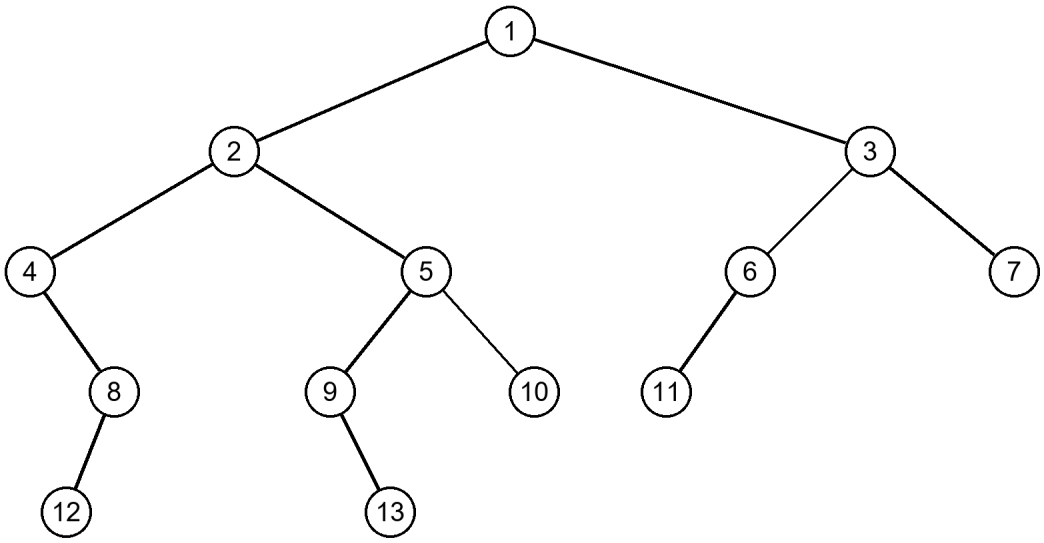
Inorder _____

Postorder _____

Lecture 10 Successor Deletion

20. Delete an internal node using its successor

Start from the original Tree A below. Delete node 2 using its inorder successor. Circles show node numbers only, not stored keys. Follow the left and right links; do not compare node numbers. Hidden keys obey the BST ordering rule. For these drawings, move the successor or predecessor node into the deleted node position, retain its number, and reconnect its remaining child. Do not copy or change node numbers.



Original Tree A fresh start

Successor node number _____ Its child reconnects to _____

Draw the complete resulting tree and mark every changed link.

21. Locate a successor

For a node that has two children, where is its inorder successor?

A. At the maximum-key node in its left subtree.

B. Always at its immediate right child.

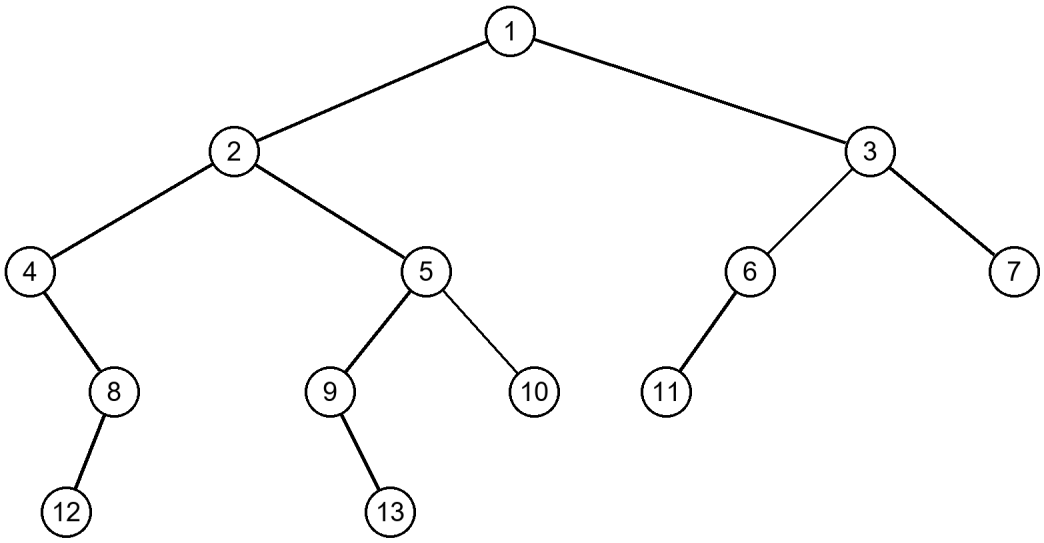
C. At the minimum-key node in its right subtree.

D. Always at its parent.

Lecture 10 Predecessor Deletion

22. Delete an internal node using its predecessor

Independently restart from original Tree A. Delete node 2 using its inorder predecessor. Circles show node numbers only, not stored keys. Follow the left and right links; do not compare node numbers. Hidden keys obey the BST ordering rule. For these drawings, move the successor or predecessor node into the deleted node position, retain its number, and reconnect its remaining child. Do not copy or change node numbers.



Original Tree A independent start

Predecessor node number _____ Its child reconnects to _____

Draw the complete resulting tree and mark every changed link.

23. BST ordering invariant

Which condition is required at every node in a BST with unique keys?

A. Only its immediate children must be ordered correctly.

B. Every left-subtree key is smaller and every right-subtree key is larger.

C. Its two subtrees must have equal heights.

D. Its node number must be smaller than both child numbers.

Lecture 10 Expression Tree Construction

24. Draw an expression tree from postfix

Draw the binary expression tree for a b c * + d e - /. Show operators at internal nodes and operands at leaves. Preserve operand order. Then write the fully parenthesized infix expression and state which traversal produces the original postfix sequence.

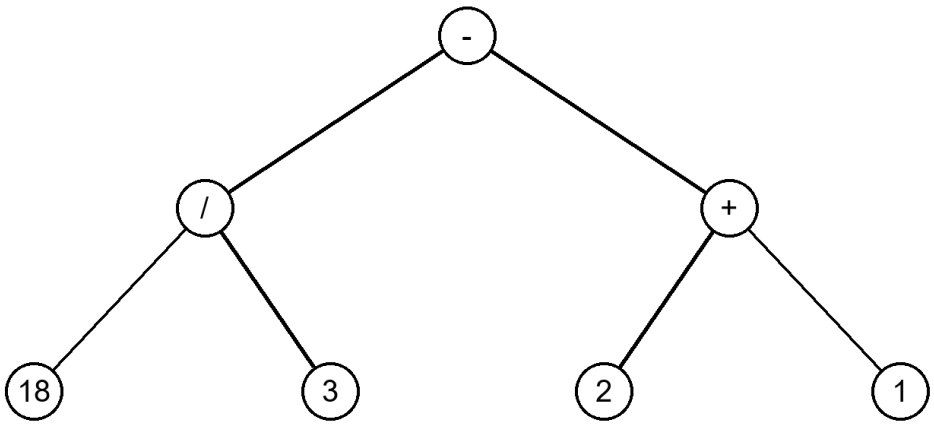
Infix _____

Traversal producing postfix _____

Expression Trees Reading and Drawing

25. Read and evaluate an expression tree

The circles contain operators and integer operands. Give the fully parenthesized infix expression, prefix expression, postfix expression, and final numerical value. Use the drawn left and right operand order.



Practice expression tree operators and operands

Infix _____

Prefix _____

Postfix _____

Value _____

26. Construct from infix

Draw the expression tree for (p - q) * (r + s / t). Multiplication and division have higher precedence than addition and subtraction. Label all operands and operators. Then give the postfix expression.

Postfix _____

Expression Trees Traversal and Stack Construction

27. Traversal and operand order

Which statement is correct for a binary expression tree?

A. Preorder yields postfix, and swapping subtraction operands never changes the result.

B. Inorder without parentheses always preserves the original grouping.

C. Postorder yields postfix, and the right operand subtree must retain its position for subtraction and division.

D. Every expression tree must satisfy BST key ordering.

28. Build with a stack of subtrees

Process the postfix expression a b c * + d -. Push an operand as a one-node tree. At an operator, pop the right subtree first and the left subtree second, then push the combined tree. In the table, represent each subtree by its fully parenthesized infix expression. Show the stack bottom to top after each token, then draw the final tree.

Token	Stack of subtrees bottom to top
a	
b	
c	
*	
+	
d	
-	

Lecture 11 Priority Queues and Max Heaps

29. Removal order and ties

Which statement is correct?

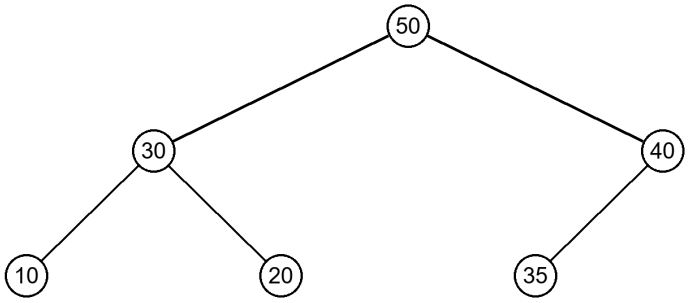
A. A FIFO queue removes the largest-priority item first.

B. Java PriorityQueue guarantees arrival order among equal priorities.

C. A max-priority queue removes the smallest item first.

D. A max-priority queue removes a greatest-priority item; equal-priority stability needs an explicit tie rule.

30. Heap properties and array positions



Max heap represented by [50, 30, 40, 10, 20, 35]

State the complete-tree shape property and the max-heap ordering property. For a zero-based array, give the left-child and right-child indexes for index i, and the parent index for i > 0. Explain why this is not a BST.

31. Max heap insertion

Insert 45 into the pictured heap. Show the appended array and each sift-up swap with its indexes. Write the final level-order array, then sketch the final heap in the space below.

Stage	Array and swap indexes
After append	
After swap	
Final array	

Lecture 11 Max Heap Deletion

32. Remove the maximum

Use the result of Question 31. Remove the maximum, state the returned value, and show root replacement and every sift-down swap. Draw the final heap. Choose the larger child when sifting down.

Returned value _____

Stage	Array and swap indexes
Root replacement	
After swap 1	
After swap 2 if needed	
Final array	

33. Bottom-up heap construction

Build a max heap from [12, 90, 60, 45, 25, 100, 36, 85]. Process internal indexes 3, 2, 1, 0 and sift down each. Show the complete array after each rebuild. No code is required.

Rebuilt index	Array after rebuild
3	
2	
1	
0	

Bottom-up build cost _____ Worst-case cost of n separate insertions _____

Lecture 11 Heap Analysis

34. Heights and operation costs

Count height in edges. Give the heights of complete heaps containing 16, 17, and 512 nodes, and the maximum number of nodes in a binary tree of height 5. State peek cost and sift-up/sift-down cost, separating any array-resize cost.

Quantity	Answer
Height for 16 nodes	
Height for 17 nodes	
Height for 512 nodes	
Maximum nodes at height 5	
Peek and repair costs	

35. Min-priority queue trace

An empty min-priority queue receives insert 3, insert 1, insert 2, remove-min, peek-min. Give the two returned values and the remaining priorities. Is iterating over an ordinary heap representation guaranteed to list priorities in sorted order?

36. Recognize a max heap

Which zero-based level-order array satisfies max-heap ordering? Assume each array represents a complete shape.

A. [50, 30, 40, 10, 20, 35]

B. [30, 50, 40, 10, 20, 35]

C. [50, 30, 40, 10, 45, 35]

D. [50, 30, 40, 10, 20, 55]

Lecture 12 Hashing Concepts

37. Hash codes and indexes

A mathematical modulo operation maps hash code -11 to an index for capacity 7. Which gives the nonnegative index and the reason an absolute-value shortcut can fail for a Java int?

A. Index -4; array indexes may be negative.

B. Index 4; absolute value is always safe.

C. Index 3; the most negative int has no positive int counterpart.

D. Index 0; all negative hashes map to zero.

38. Collisions and load factor

A table has 11 buckets and 6 stored entries. Define a collision, distinguish a hash code from a table index, and compute the load factor. Then complete the comparison below.

Collision method	What happens after a collision
Separate chaining	
Linear probing	
Quadratic probing	
Double hashing	

For open addressing, explain why a search must follow the same probe sequence used during insertion.

Lecture 12 Double Hashing Trace

39. Insert with double hashing

Use an empty table of capacity 11, indexed 0 through 10, with no resizing. Let h1(k) = k mod 11, h2(k) = 7 - (k mod 7), and probe(k, i) = (h1(k) + i * h2(k)) mod 11, starting at i = 0. Insert 22, 1, 13, 11, 24, 33 in that order. Include the successful index in every probe sequence.

Key	h1	h2	Probed indexes	Final index
22				
1				
13				
11				
24				
33				

Index	0	1	2	3	4	5	6	7	8	9	10
Key											

40. Double-hashing coverage

What property must the step size have to visit every slot in a table of capacity m?

A. The step must equal zero.

B. The step must be relatively prime to m.

C. The step must be even.

D. Any nonzero step works with every capacity.

Explain why the capacity-11 configuration above meets this property.

Lecture 12 Deletion Chaining and Rehashing

41. Tombstone search

Use the final double-hashing table from Question 39. Delete key 13 by marking its slot DELETED. Search for 33. Give the deleted index and every search probe. Explain why marking the deleted slot EMPTY could give a false absence result. Also explain why an insertion may need to continue past a tombstone before reusing it.

42. Separate chaining

Insert 10, 15, 7, 12, 20 into capacity 5 using key mod 5. Append to each bucket chain. Draw or list each chain, then give the load factor and state whether chaining permits a load factor above 1.

Bucket	Chain in insertion order
0	
1	
2	
3	
4	

43. Rehash into a larger table

Insert 22, 1, 13, 11, 24, 33, in that order, into an empty capacity-23 table using key mod 23 and linear probing. List the final index of each key. Explain why copying its old index is incorrect.

Key	22	1	13	11	24	33
New index						

44. Trace a polynomial hash

Starting with h = 0, update h to 31 times h plus the character code. For ab, use character codes a = 97 and b = 98. Show h after each character, then map the final hash to an index modulo 7. Do arithmetic only; do not write a method.

Lecture 13 HashSet Integer Insertions

Use the standard Java HashSet<Integer> ADT. Integer objects compare by their integer values. Show mathematical contents; no iteration order is assumed. No coding is required.

45. Repeated integer values

Start with an empty HashSet<Integer>. Insert Integer objects with values 10, 20, 10, 30, 20 in order. Give the contents and size after every insertion. Explain why inserting an existing value does not increase the size.

Inserted value	HashSet contents	Size
10		
20		
10		
30		
20		

Final HashSet size _____

---

46. Distinct objects with equal integer values

Start with another empty HashSet<Integer>. Objects I1, I2, I3, I4, and I5 are distinct objects. Their integer values are shown below. Insert them in order. Record the stored integer values and set size. Does different object identity force a new set entry? Explain.

Inserted object and value	Stored integer values	Size
I1 has value 1000		
I2 has value 1000		
I3 has value 2000		
I4 has value 1000		
I5 has value 2000		

Final HashSet size _____

Lecture 13 HashSet Collisions and Sizes

47. Collision does not mean equality

An empty HashSet<Integer> uses bucket index k mod 11 for this exercise. Insert values 12, 23, 34, 12. These values all reach bucket 1. Give the contents and size after each insertion. State whether the collision makes 12, 23, and 34 equal.

Inserted value	HashSet contents	Size
12		
23		
34		
12		

Final HashSet size _____

48. Two sets and their result sizes

Start with empty HashSet<Integer> objects S and T. Insert 1, 2, 2, 3 into S and 3, 4, 3 into T. List the contents and size of each set. Independently form a new set for each operation below, leaving S and T unchanged.

ADT or result	Contents	Size
S		
T		
S union T		
S intersection T		
S minus T		

Explain what size counts in a HashSet<Integer>.

Lecture 13 HashMap Integer Insertions

Use the standard Java HashMap<Integer, Integer> ADT. Each pair is (key, value). Keys are unique; inserting an existing key replaces its associated value. Size counts key-value mappings. No iteration order is assumed.

49. Repeated keys

Start with an empty HashMap<Integer, Integer>. Insert the pairs below in order. Give every mapping and the map size after each insertion. Explain why replacing a value for an existing key does not add an entry.

Inserted key-value pair	HashMap contents	Size
(1, 10)		
(2, 20)		
(1, 30)		
(3, 20)		
(2, 40)		

Final HashMap size _____

---

50. Different keys with the same value

Start with another empty HashMap<Integer, Integer>. Insert (10, 7), (20, 7), (30, 7), (10, 9) in order. Trace the mappings and size. At the end, give the number of keys and the number of distinct associated values. Can multiple keys share the same value?

Inserted key-value pair	HashMap contents	Size
(10, 7)		
(20, 7)		
(30, 7)		
(10, 9)		

Final map size _____ Number of distinct values _____

Lecture 13 Compare Set and Map Sizes

51. HashMap and its key-set view

Start with an empty HashMap<Integer, Integer>. Insert (4, 40), (5, 50), and (6, 60), then obtain its backed key-set view. Next reinsert (5, 55), then remove integer key 5 through that view. Complete the table and state whether the view is an independent HashSet copy.

Stage	HashMap contents	Map size	Key-set contents	Key-set size
After obtaining the view				
After reinserting (5, 55)				
After removing 5 through the view				

---

52. Same integers in a HashSet and a HashMap

Start with empty HashSet<Integer> S and HashMap<Integer, Integer> M. Process the values 5, 5, 8, 5, 8, 9 in order. At step j, insert the Integer value into S and insert (value, j) into M, where j starts at 1. Show the contents and size of both ADTs after each step. Explain why both sizes count distinct integer values here, even though the map stores pairs.

Step and value	HashSet contents	Set size	HashMap contents	Map size
1: 5				
2: 5				
3: 8				
4: 5				
5: 8				
6: 9				

Final HashSet size _____ Final HashMap size _____
